

Technical Documentation

1. Introduction

Retrieval-Augmented Generation (RAG) is an architecture that combines information retrieval with answer generation. Instead of asking a model to answer from parametric memory alone, the system first retrieves relevant source material and then generates a grounded answer from that material.

RAG is needed in customer support because policy accuracy matters. Refund rules, billing guidance, warranty terms, and escalation procedures change over time. A retrieval layer allows the assistant to answer from the current knowledge base instead of relying on stale model memory.

This project uses a customer-support assistant as the target use case. The assistant answers policy and troubleshooting questions from a PDF handbook and escalates risky cases to a human reviewer.

2. System Architecture Explanation

The system is divided into two flows:

- ingestion flow: PDF -> extraction -> chunking -> embedding -> ChromaDB
- query flow: user question -> retrieval -> routing -> answer or HITL

The document ingestion pipeline prepares the knowledge base once and stores reusable embeddings. The query-time workflow focuses on low-latency retrieval and deterministic routing. LangGraph manages the state transitions so the orchestration logic remains explicit and maintainable.

The architecture contains these collaborating components:

- document loader
- chunker
- embedder
- vector store
- retriever
- response generator
- routing layer
- graph workflow
- HITL ticketing module

3. Design Decisions

Chunk Size Choice

The implementation uses a chunk size of about 650 characters with 120 characters of overlap. This is a compromise:

- smaller chunks improve retrieval precision

- larger chunks preserve more context
- overlap reduces information loss at chunk boundaries

Embedding Strategy

The implementation uses deterministic local embeddings to keep the project runnable without external model downloads or API credentials. The interface design keeps the embedder replaceable, so a semantic embedding model can be added later without changing retrieval or workflow code.

Retrieval Approach

Top-k vector similarity search is used because it is simple, explainable, and effective for short policy-style knowledge-base chunks. The returned scores are also used as confidence signals for routing.

Prompt and Answer Design Logic

The answer generator is intentionally grounded. It ranks context sentences by lexical overlap with the query and builds a concise response with citations. This keeps the demo deterministic while preserving the RAG pattern.

4. Workflow Explanation

LangGraph is used to model the support assistant as a stateful workflow rather than a monolithic function call.

Node Responsibilities

- ``process_query`` handles intent detection, retrieval, confidence estimation, and draft answer generation
- ``human_review`` creates escalation tickets or accepts a human override
- ``output_response`` emits the final answer text

State Transitions

The workflow starts with the user query in state. The processing node enriches state with intent, retrieved chunks, confidence, and an answer draft. A conditional edge then chooses the answer branch or HITL branch. The final node formats the response returned to the caller.

5. Conditional Logic

Intent Detection

Intent detection is currently rule-based and optimized for the included support handbook. It maps keyword patterns into domains such as returns, billing, troubleshooting, account access, and order status.

Routing Decisions

Routing decisions combine:

- top retrieval score

- presence or absence of retrieved chunks
- sensitive escalation keywords
- multi-issue query signals

This design prevents the assistant from over-answering when context quality is weak.

6. HITL Implementation

Human intervention is part of the normal system design rather than an afterthought.

Role of Human Intervention

Human reviewers handle requests that need judgment, compliance awareness, exception handling, or de-escalation. Typical examples include chargebacks, legal threats, data disputes, and unclear multi-part questions.

Benefits

- reduces the risk of confidently wrong answers
- creates an auditable path for sensitive requests
- allows safe deployment before full automation confidence is reached

Limitations

- adds handling latency
- requires an operational review queue
- can become a bottleneck if routing thresholds are too conservative

7. Challenges and Trade-offs

Retrieval Accuracy vs Speed

Higher retrieval depth may improve recall but increases post-processing and answer latency.

Chunk Size vs Context Quality

Very small chunks can miss surrounding policy nuance. Very large chunks may dilute retrieval relevance.

Cost vs Performance

A hosted LLM or embedding service usually improves semantic quality but increases operating cost and introduces credential and network dependencies.

8. Testing Strategy

The project includes lightweight unit tests for deterministic embeddings and intent classification. Practical verification should also include scenario tests that cover:

- direct answer queries

- low-confidence queries
- policy-sensitive escalation queries
- human override resolution

Sample Queries

- "How long do refunds take after my return is received?"
- "My package says delivered but I still do not have it. What should I do?"
- "I was charged twice and I want to open a chargeback."
- "I lost access to my email and also need to cancel my subscription."

9. Future Enhancements

- multi-document ingestion with document versioning
- semantic re-ranking on top of vector retrieval
- user feedback loop for answer quality tuning
- session memory for ongoing support conversations
- API or web deployment for broader use
- stronger analytics around escalation reasons and retrieval confidence

Implementation Notes

The implementation is intentionally modular. The local embedder and response synthesizer make the project self-contained for evaluation, while the boundaries around embedding and generation make it straightforward to introduce production-grade model providers later.